

Algoritmos Cuánticos

Algoritmos



Contreras Reyes Orlando
García Barrera Iker Emiliano
Asesor: José Luis López González

Universidad Autónoma de Aguascalientes
Departamento de Sistemas Electrónicos



31 de julio de 2026

1 Suma

2 DFT y FFT

- Muestreo y Teorema de Nyquist
- Muestreo y Teorema de Nyquist
- Raíces de la unidad
- Aplicaciones

3 Conclusiones

Algoritmo de la suma

El algoritmo de la suma es uno de los fundamentos básicos. Consiste en tomar dos objetos **del mismo tipo** y combinarlos para obtener un resultado que representa la adición de ambos, además el algoritmo **esta sujeto** a las reglas de la aritmética del tipo de objeto que se está sumando

Ejemplo

Sumar dos números enteros: $5 + 3 = 8$.

Sumar dos vectores: $\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$.

Sumar dos bits: $1 + 1 = 10$.

Del Mundo Continuo al Mundo Discreto

Problema

El mundo real es continuo, pero los microcontroladores procesan señales **discretas**.

Solución: Muestreo de señales

¿Por qué discretizar?

Las señales en el mundo real son continuas, pero los sistemas digitales solo procesan muestras discretas. El muestreo es el proceso de convertir una señal continua $f(t)$ en una secuencia discreta $f[n] = f(nT_s)$, donde T_s es el período de muestreo.

$$\int_{-\infty}^{\infty} f(t) e^{-j\omega t} dt \implies \sum_{n=0}^{N-1} f[n] e^{-j\omega t}$$

Muestreo y Teorema de Nyquist

Teorema de Nyquist-Shannon

Para reconstruir sin pérdida una señal continua a partir de muestras, la frecuencia de muestreo f_s debe ser al menos el doble de la máxima frecuencia presente en la señal:

$$f_s > 2f_{\text{máx}}$$

Herramienta web para entender el teorema de Nyquist:

<https://resources.nerdfirst.net/sampling.html>

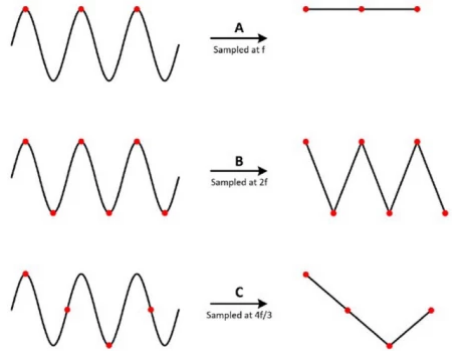


Figura: Una velocidad de muestreo demasiado baja puede causar una reconstrucción inexacta de la forma de onda.

Nomenclatura

$x[n] \triangleq$ Amplitud de la señal en el tiempo $t_n(s)$

$n \triangleq [0, N - 1]$ índice de muestra

$T_s \triangleq$ Periodo

$t_n \triangleq nT_s$ tiempo de la muestra n

$X[k] \triangleq$ Espectro de la señal x en la frecuencia radial k -ésima

$k \triangleq [0, N - 1]$ índice de frecuencia

$N \triangleq$ Número total de muestras de tiempo y frecuencia

$W_N \triangleq e^{-2\pi j/N}$ raíz primitiva N -ésima de la unidad

Constantes

Las constantes a utilizar son las siguientes:

$$j \triangleq \sqrt{-1}$$

$$\pi \approx 3,14159$$

$$e \triangleq \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n$$

$$e \approx 2,71828$$

Definicion DFT

Fase normalizada

Si $t_n = nT_s$ y la frecuencia del bin k es

$\nu_k = \frac{k}{NT_s}$, entonces:

$$\nu_k t_n = \frac{k}{NT_s} (nT_s) = \frac{kn}{N}$$

Por eso el núcleo queda como $e^{-j2\pi kn/N}$: N normaliza la fase al número total de muestras y garantiza ortogonalidad entre bins.

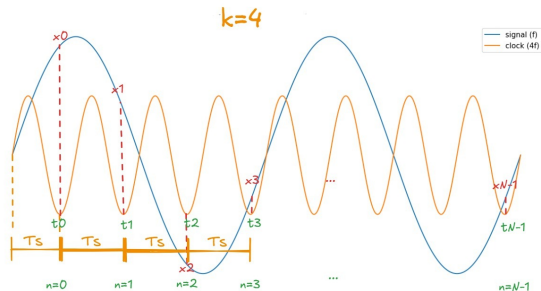


Figura: Representación visual de los elementos de frecuencia.

Definición de la DFT (Discrete Fourier Transform)

DFT - Transformada Discreta de Fourier

Dada una secuencia de N muestras $x[n]$ con $n = 0, 1, \dots, N - 1$, la DFT se define como:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad W_N = e^{-2\pi j/N}, \quad k = 0, 1, \dots, N - 1$$

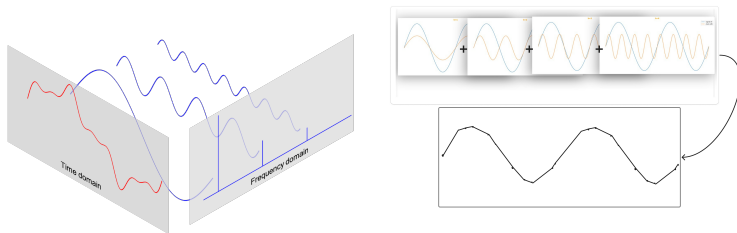


Figura: Reconstrucción de una señal a través de diferentes frecuencias (ν)

Construcción de la Matriz Desarrollando la sumatoria con un $N = 4$ se obtiene la siguiente matriz:

$$X[0] = x[0]W_4^{0 \cdot 0} + x[1]W_4^{0 \cdot 1} + x[2]W_4^{0 \cdot 2} + x[3]W_4^{0 \cdot 3}$$

$$X[1] = x[0]W_4^{1 \cdot 0} + x[1]W_4^{1 \cdot 1} + x[2]W_4^{1 \cdot 2} + x[3]W_4^{1 \cdot 3}$$

$$X[2] = x[0]W_4^{2 \cdot 0} + x[1]W_4^{2 \cdot 1} + x[2]W_4^{2 \cdot 2} + x[3]W_4^{2 \cdot 3}$$

$$X[3] = x[0]W_4^{3 \cdot 0} + x[1]W_4^{3 \cdot 1} + x[2]W_4^{3 \cdot 2} + x[3]W_4^{3 \cdot 3}$$

Pudiendo escribir esto en forma matricial como:

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^4 & W_4^9 \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix} \Rightarrow \vec{X} = \widehat{W}_4^{kn} \vec{x}$$

Definición de la DFT - Forma matricial

Generalizando para un N cualquiera podemos obtener la matriz de Fourier, lo que da lugar a una matriz de $N \times N$ con elementos que son potencias de W_N . La DFT se puede calcular multiplicando esta matriz por el vector de entrada $x[n]$, lo que da como resultado el vector de salida $X[k]$.

$$\begin{bmatrix} X_0 \\ X_1 \\ X_2 \\ \vdots \\ X_{N-1} \end{bmatrix} = \begin{bmatrix} W_N^{0 \cdot 0} & W_N^{0 \cdot 1} & W_N^{0 \cdot 2} & \dots & W_N^{0 \cdot (N-1)} \\ W_N^{1 \cdot 0} & W_N^{1 \cdot 1} & W_N^{1 \cdot 2} & \dots & W_N^{1 \cdot (N-1)} \\ W_N^{2 \cdot 0} & W_N^{2 \cdot 1} & W_N^{2 \cdot 2} & \dots & W_N^{2 \cdot (N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ W_N^{(N-1) \cdot 0} & W_N^{(N-1) \cdot 1} & W_N^{(N-1) \cdot 2} & \dots & W_N^{(N-1) \cdot (N-1)} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{N-1} \end{bmatrix} \quad (1)$$

Raíces de la unidad

Definición

Las raíces N-ésimas de la unidad son números complejos que satisfacen $z^N = 1$. En el contexto de la DFT, usamos:

$$W_N = \sqrt[N]{e^{-2\pi j}} = e^{-2\pi j/N}$$

Esta es la raíz primitiva N-ésima de la unidad. Sus potencias W_N^k para $k = 0, 1, \dots, N-1$ son todas las N raíces.

Propiedad fundamental

① Para $k \in \mathbb{Z}$

$$W_N^{kN} = 1$$

Demostración:

$$W_N^{kN} = (e^{-2\pi j/N})^{kN} = e^{-2\pi jk} = (e^{2\pi j})^{-k} = 1^{-k}$$

Esta periodicidad es la base que permite que la DFT tenga estructura.

Propiedades de las raíces de la unidad

Propiedades clave

2 Para $k \in \mathbb{Z}$:

$$W_N^{k+N} = W_N^k$$

Demostración:

$$\begin{aligned} W_N^{k+N} &= W_N^k \cdot W_N^N = e^{-2\pi j(k)/N} \cdot e^{-2\pi j(N)/N} = \\ &= e^{-2\pi jk/N} \cdot 1 = W_N^k \end{aligned}$$

3 Para $N > 0$:

$$W_N^{N/2} = -1$$

Demostración:

$$W_N^{N/2} = \left(e^{-2\pi j/N}\right)^{N/2} = e^{-2\pi j(N/2)/N} = e^{-\pi j} = -1$$

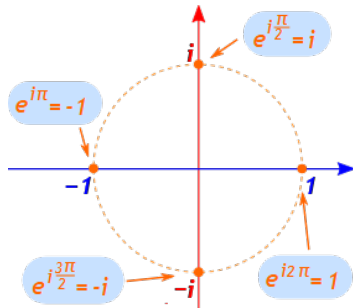


Figura: Circulo unitario con 4 raíces

Propiedades Aplicadas

Empleando las propiedades definidas previamente en la matriz con $N = 4$

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix}$$

Enfocandonos en la matriz de Fourier $\widehat{W}_4^{kn}$

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^6 \\ 1 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \xrightarrow[\text{propiedad 2}]{W_4^6=W_4^2, W_4^9=W_4^1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^2 \\ 1 & W_4^3 & W_4^2 & W_4^3 \end{bmatrix} \xrightarrow[\text{propiedad 3 y 1}]{W_4^2=-1, W_4^4=1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & W_4^3 \\ 1 & -1 & 1 & -1 \\ 1 & W_4^3 & -1 & W_4^1 \end{bmatrix}$$

Quedando en solo dos W por computar $1W_4^1$ y W_4^3 .

FFT - Definición y Problema

Definición

La FFT (*Fast Fourier Transform*) es un algoritmo eficiente para calcular la Transformada Discreta de Fourier (DFT).

DFT

La DFT convierte una señal en el dominio del tiempo al dominio de la frecuencia:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

Problema

Calcular la DFT directamente requiere:

$$O(N^2)$$

operaciones.

Idea de la FFT

La FFT divide la DFT en problemas más pequeños usando simetrías y periodicidad:

$$O(N \log_2 N)$$

Ventajas

- Reduce drásticamente el tiempo de cálculo.
- Permite procesamiento digital en tiempo real.
- Muy utilizada en:
 - Audio
 - Comunicaciones
 - Imágenes
 - Radar
 - Análisis espectral

Algoritmo de Cooley-Tukey

Principio de divide y conquista

El algoritmo de Cooley-Tukey divide la DFT de N puntos en dos DFTs de $N/2$ puntos:

Índices pares e impares

$$X[k] = \sum_{n=0}^{N/2-1} x[2n]e^{-j2\pi k(2n)/N} + \sum_{n=0}^{N/2-1} x[2n+1]e^{-j2\pi k(2n+1)/N}$$

Simplificación

$$X[k] = E[k] + W_N^k O[k]$$

donde:

- $E[k]$ = FFT de muestras pares
- $O[k]$ = FFT de muestras impares
- $W_N^k = e^{-j2\pi k/N}$ = factor de giro (*twiddle factor*)

Este proceso se repite recursivamente hasta llegar a DFTs de 2 puntos.

Operaciones Butterfly - Teoría

¿Qué es una mariposa?

La operación butterfly es la unidad computacional fundamental de la FFT. Combina dos números complejos.

Estructura básica

Entrada: a y b

Salida:

$$y_1 = a + W \cdot b$$

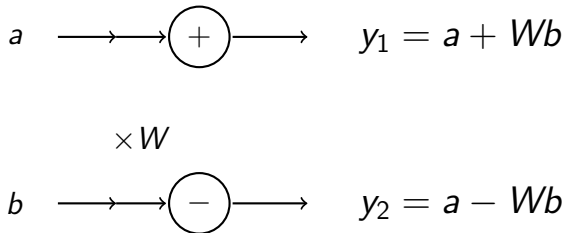
$$y_2 = a - W \cdot b$$

donde $W = e^{-j2\pi k/N}$ es el factor de giro.

Interpretación

- W es el *twiddle factor* (factor de rotación)
- La operación es lineal y se repite recursivamente
- Fundamental para el algoritmo de Cooley-Tukey

Operaciones Butterfly - Diagrama de Flujo



Características principales

- La entrada superior pasa directamente a la suma
- La entrada inferior se multiplica por el factor W antes de las operaciones
- Dos operaciones: suma y resta (resta equivale a suma con signo negativo)
- Implementación eficiente: **1 multiplicación + 2 sumas complejas**

Operaciones por butterfly

Cada operación butterfly requiere:

- 1 multiplicación compleja: $W \cdot b$
- 2 sumas complejas: $(a + W \cdot b)$ y $(a - W \cdot b)$

Equivalencia en operaciones reales

- 1 multiplicación compleja = 4 multiplicaciones reales + 2 sumas reales
- 2 sumas complejas = 4 sumas reales

Total por butterfly: **4 multiplicaciones y 6 sumas reales**

Eficiencia de la FFT

Número total de butterflies

Para una FFT de N puntos (donde $N = 2^m$):

$$\text{Butterflies} = \frac{N}{2} \log_2 N$$

Comparación de complejidad

- **DFT directa:** N^2 multiplicaciones
- **FFT:** $\frac{N}{2} \log_2 N$ multiplicaciones
- **Ganancia:** ~ 1000 veces más rápida para $N = 2048$

Ejemplo numérico

Para $N = 1024 = 2^{10}$:

- DFT: $1024^2 = 1,048,576$ operaciones
- FFT: $\frac{1024}{2} \times 10 = 5,120$ operaciones

Puntos clave

- La FFT reduce la complejidad de $O(N^2)$ a $O(N \log N)$
- Las operaciones butterfly son la base del algoritmo
- Cooley-Tukey utiliza el principio divide y conquista
- Esencial en aplicaciones de procesamiento de señales en tiempo real

¿Por qué es importante?

Sin la FFT, muchas aplicaciones modernas (streaming de audio, teleconferencias, procesamiento de imágenes) serían computacionalmente impracticables.

- **Procesamiento de audio:** análisis de tonos, compresión (MP3)
- **Procesamiento de imágenes:** compresión (JPEG), reconocimiento de patrones
- **Análisis de espectro:** identificar componentes de frecuencia en señales
- **Filtrado digital:** diseño e implementación de filtros
- **Telecomunicaciones:** modulación, multiplexación
- **Ciencia:** resonancia magnética, espectroscopia, astronomía

Conclusión

La FFT es una de las herramientas computacionales más importantes del siglo XX. Sin ella, muchas tecnologías modernas no serían viables.

DFT vs FFT

Cálculo directo (DFT)

- Complejidad: $\mathcal{O}(N^2)$
- Lento para N grande
- Conceptualmente simple
- Raras veces usado en práctica

Algoritmo rápido (FFT)

- Complejidad: $\mathcal{O}(N \log N)$
- Orden de magnitud más rápido
- Explota simetría y periodicidad de W_N
- Estándar en procesamiento digital

Ejemplo

Para $N = 1024$: DFT requiere ~ 1 millón de operaciones, FFT solo ~ 10000 . ¡100 veces más rápida!

- ➊ **Transformadas Integrales:** Herramientas para cambiar de dominio
- ➋ **Transformada de Fourier:** Convierte señales del dominio temporal al de frecuencia
- ➌ **Propiedades:** Linealidad, desplazamiento, escalamiento, convolución
- ➍ **DFT:** Versión discreta para procesamiento digital
- ➎ **FFT:** Algoritmo eficiente ($O(N \log N)$) que revolucionó el procesamiento de señales

Referencias Principales

-  Bracewell, R. *The Fourier Transform and Its Applications*, 3rd ed. McGraw-Hill, 1999.
-  Wolfram MathWorld. *Discrete Fourier Transform*.
<https://mathworld.wolfram.com/DiscreteFourierTransform.html>
-  GeeksforGeeks. *Fourier Transform*.
<https://www.geeksforgeeks.org/maths/fourier-transform/>